

# DOCCLASS-MERGED → Mailroom Ingress & Evaluation Corpus

## Comprehensive Implementation Plan

**Repository:** Exios66/mailroom-dev **Dataset:** Lucius-Morningstar/docclass-merged  
**Purpose:** Mailroom document-ingress simulation, classification, extraction, grouping, routing, adjudication, retry, and regression evaluation **Status:** Current working corpus  
**Primary target:** Harden and extend the current corpus without prematurely turning it into a generic benchmark or training dataset.

**Grounded against the live repo** (fetched 2026-09-02): `README.md` , `AGENTS.md` , `docs/v7-taxonomy.md` . Where earlier drafts of this plan guessed at taxonomy or workflow details, those guesses are corrected below and flagged inline as `[corrected]` . New material not present in earlier drafts is flagged `[new]` . Anything unflagged is unchanged from the original draft and has not been independently re-verified against the live repo in this pass.

---

## 0. Glossary & Canonical Vocabulary

`[new]` For anyone — human or agent — picking this up cold. Terms are drawn from the live repo, not invented here.

| Term       | Meaning                                                                                                                                          |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Mailroom   | The LangGraph multi-agent pipeline ( <code>packages/llm-mailroom</code> ) that ingests, classifies, routes, extracts, and adjudicates documents. |
| Sorter     | The classification stage/prompt that assigns <code>document_type</code> and <code>expected_subclass</code> .                                     |
| Specialist | The extraction agent for a given document class (e.g. contracts specialist, insurance claims specialist).                                        |
| Judge      | Scores a specialist’s extraction as complete or partial.                                                                                         |
| Arbiter    | Resolves a partial/failed judge result — stands, orders re-extraction, or escalates to human review.                                             |

|                                               |                                                                                                                                                                  |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dojo</b> ( <code>llm-dojo-scoring</code> ) | The shared deterministic scoring/error-analysis engine used across the pipeline.                                                                                 |
| <b>Matter</b>                                 | The real-world legal/insurance matter a document belongs to — a grouping concept, not a document itself.                                                         |
| <b>Group</b>                                  | A logical document bundle within a matter (e.g. contract + amendment + exhibit).                                                                                 |
| <b>GEPA</b>                                   | The prompt-optimization/versioning loop run in <code>packages/llm-entity-extraction</code> over CUAD/LegalBench/MAUD.                                            |
| <b>v7</b>                                     | The current corpus/schema revision of <code>docclass-merged</code> — a corpus revision label, <b>not</b> a taxonomy name (see §5).                               |
| <b>HUB_CLASSES</b>                            | The literal source-of-truth constant ( <code>packages/llm-mailroom/src/pipeline/hf_corpora.py</code> ) listing which document classes the Hub corpus represents. |
| <b>GT</b>                                     | Ground truth.                                                                                                                                                    |
| <b>OOD</b>                                    | Out-of-distribution — a document that shouldn't map to any of the five live classes.                                                                             |
| <b>Observatory</b>                            | The Langfuse-backed trace/observability surface behind the <code>The-Mailroom</code> visualizer.                                                                 |

## 1. Mission

`docclass-merged` is not a model-training dataset.

It is the controlled document universe used to simulate documents arriving at the Mailroom.

The corpus exists to answer: Can Mailroom correctly ingest heterogeneous legal, corporate, correspondence, and insurance documents; classify them; route them to the correct specialist; extract the expected information; group related documents; detect failures; escalate appropriately; incorporate adjudication; retry processing; and produce the correct final record?

The dataset's multi-source composition is therefore intentional. Do not "fix" the fusion of CUAD, MAUD, corporate records, Enron correspondence, and CMS DE-SynPUF merely because those sources do not form a statistically natural population. They represent

different document families that Mailroom needs to handle.

---

## 2. Current Repository Architecture Must Be Treated as Canonical

Before changing the dataset, inspect and respect:

- `packages/llm-mailroom` — LangGraph pipeline; `src/pipeline/hf_corpora.py` defines `HUB_CLASSES`, the **actual** source of truth for which document classes the live Hub corpus represents [corrected]
- `packages/llm-entity-extraction` — prompt-experiment loop (GEPA prompt versions over CUAD/LegalBench/MAUD)
- `packages/llm-dojo-scoring` — shared deterministic scoring engine
- `packages/local-mailroom-sandbox`
- `packages/claims-data-eda` — CMS DE-SynPUF EDA feed
- `packages/Enron-Evaluation-Environment` — Enron correspondence EDA feed
- `packages/mailroom-corpus-eda` — docclass-merged corpus EDA (P0–P6) **and** the centralized HF upload helpers ( `hf_interface`, `dataset_export`, `docclass_uploader`, `intent_backfill` ) [new — missing from the original inventory; this package owns all HF publishing]
- `docs/v7-taxonomy.md` — canonical taxonomy contract for the v7 corpus; **this is the authoritative source for §5 below**, not a generic `config/taxonomy.yaml` [corrected]
- `docs/assets/mailroom-pipeline.svg` — canonical 13-node pipeline diagram
- `AGENTS.md` **and** `governance/TASKS.md` — workspace rules and the cross-agent task board; read `governance/TASKS.md` **FIRST** every session [new]
- sorter classification / subtype definitions
- specialist extraction schemas
- pilot evaluation manifests

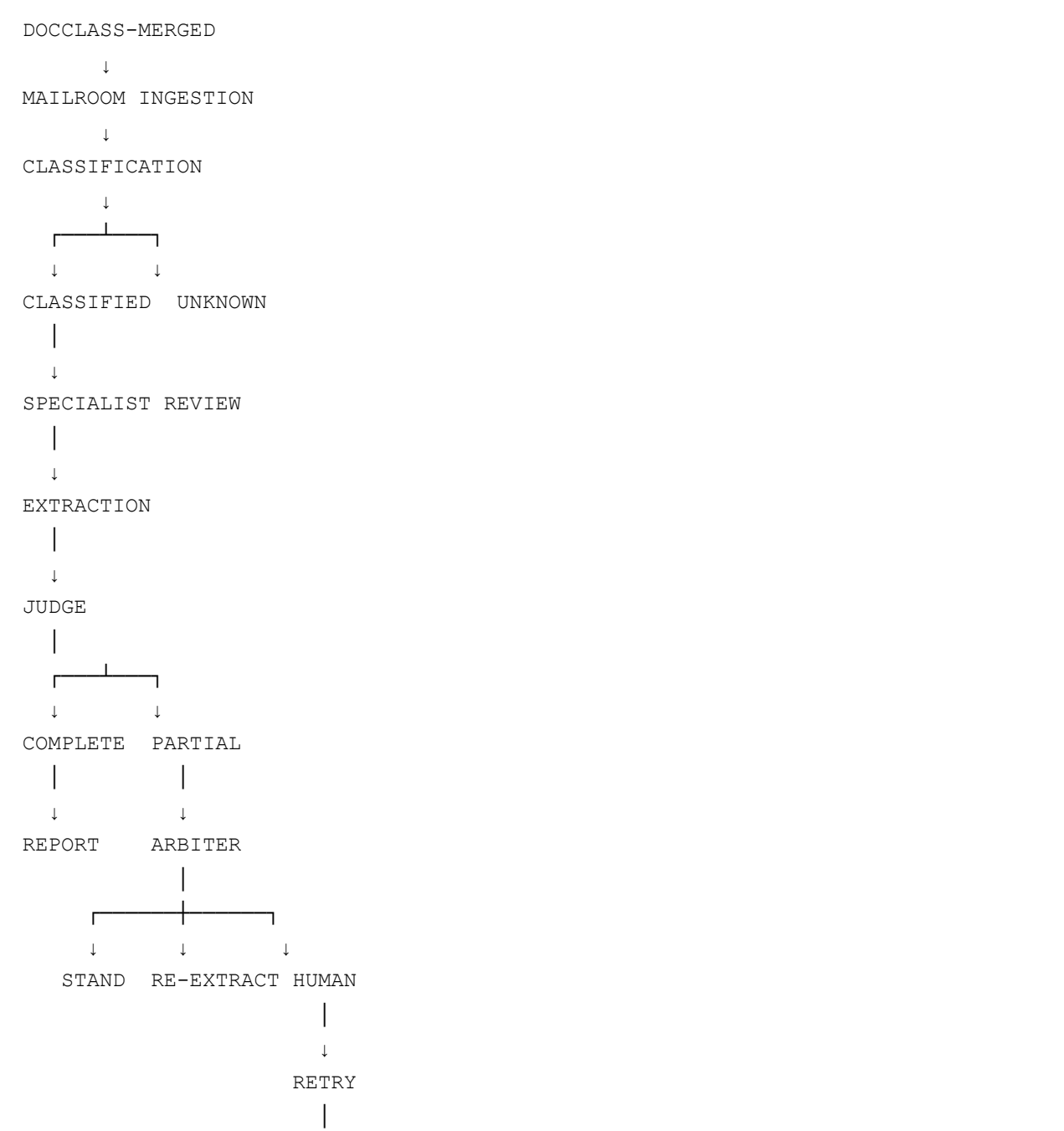
If a `config/taxonomy.yaml` or similar file exists downstream in a specialist-routing config, treat it as a *consumer* of `HUB_CLASSES`, not the source of truth — confirm which direction that dependency actually runs before writing the parity tests in §65/§65A.

The Mailroom already uses a 13-node LangGraph pipeline covering ingestion, classification, retry, reviewer, extraction, judge, arbiter, human review, boss escalation, reporting, cataloging, and archival.

Do not create a parallel dataset architecture that duplicates those concepts. The dataset should feed and test the architecture that already exists.

### 3. The Dataset's Correct Conceptual Model

The corpus should be understood as a pipeline:





The corpus therefore needs to support pipeline-state evaluation, not merely label comparison.

---

## 4. Do Not Destroy the Current Corpus

Create an immutable baseline snapshot.

**Recommended release marker:** `docclass-merged-v0.1-working`

Record:

- exact HF revision
- document count
- class counts
- split counts
- schema
- source inventory
- taxonomy revision
- annotation revision
- builder revision
- known limitations

Create:

- `docs/reports/audits/docclass_merged_baseline.md`
- a machine-readable manifest

The current working corpus must remain reproducible after all subsequent modifications.

---

## 4A. Data Governance, Licensing & PII Handling

[new] This section didn't exist in the original draft and should have from the start — the corpus fuses sources with materially different legal and privacy postures.

**Per-source licensing.** Before any redistribution, re-export, or new HF revision push, confirm and record the license terms for each fused source individually:

- CUAD (contract clause extraction) and MAUD (M&A dataset) — check current license/attribution terms at the source before re-publishing derived spans.
- Corporate records — confirm provenance/jurisdiction of any real (non-synthetic) filings included.
- Enron correspondence — this is real email from real, named individuals released via litigation discovery. It is widely used in research but is **not** license-equivalent to synthetic or purpose-built data. Do not treat it as freely re-distributable without the same care the rest of the NLP community applies to it.
- CMS DE-SynPUF — synthetic by design, but confirm current CMS terms of use before redistribution and keep its `synthetic` provenance flag mandatory, never optional (see §20, §39).

**PII handling for correspondence.** Enron rows may contain real names, real email addresses, and real (if dated) personal/financial details:

- Do not “enrich” Enron rows with additional real-world identifying detail (e.g. joining against external people-search data) under any circumstances.
- If any redaction or pseudonymization pass exists or is added, record it as its own `annotation_provenance.method` (§43) so downstream consumers know a row has been altered from source.
- Treat `source_native` Enron PII as a reason to be conservative about which fields become public-facing dataset-card examples (§78) — pick illustrative examples from synthetic or already-redacted rows where possible.

**Sensitive fields in insurance data.** Even where CMS DE-SynPUF is synthetic, downstream expansions (§36) that add real carrier correspondence or real claim documents will contain genuine PII/PHI-shaped fields (claim numbers, policy numbers, diagnosis-adjacent free text). Any future non-synthetic insurance addition needs the same redaction/provenance treatment as Enron, not a lighter one just because the current synthetic baseline is clean.

**Release gate.** Add to §91 (Release Gates): a release fails validation if any row lacks a resolvable license/provenance chain, or if a non-synthetic, non-previously-cleared source is

added without an explicit PII review note in Evidence (§84B).

## 5. Primary Taxonomy: Reconcile It With the Actual Mailroom

[corrected] This section’s premise in earlier drafts was wrong, and is fixed here. The original assumption was that a broader “extended Mailroom taxonomy” exists with several classes not yet in the corpus. The live repo’s own taxonomy contract ( docs/v7-taxonomy.md ) says otherwise: **the live Mailroom taxonomy and the v7 corpus are already aligned at exactly five classes**. There is no larger live taxonomy waiting to be caught up to.

**The actual current state (per docs/v7-taxonomy.md )**

**Live Mailroom document classes** (production pipeline) — five, no more:

```
contract
merger_agreement
corporate_record
correspondence
insurance_claim
```

**v7 represented document classes** (docclass-merged corpus) — the same five:

```
contract
merger_agreement
corporate_record
correspondence
insurance_claim
```

compliance\_filing has been **retired** from the pipeline. court (court\_opinion) and dd (due\_diligence) are **retired** and are not v7 classes. None of the three are “extended taxonomy awaiting corpus coverage” — they are former classes that were cut. Treat them as historical/changelog vocabulary only (§60, corrected accordingly).

**Canonical naming rules (from docs/v7-taxonomy.md §5 — use these exact phrasings)**

| Use                    | Meaning                                                   |
|------------------------|-----------------------------------------------------------|
| “v7 five-class corpus” | The represented document-class surface in docclass-merged |
| “five-class live       | The current production taxonomy                           |

|                              |                                                                                                  |
|------------------------------|--------------------------------------------------------------------------------------------------|
| Mailroom taxonomy"           |                                                                                                  |
| "class × subclass strata"    | The v7 stratification (document class crossed with <code>expected_subclass</code> )              |
| "correspondence intent"      | The intent dimension (see §21)                                                                   |
| "27-key ground-truth schema" | The current extraction/GT field surface, where that specific schema is being discussed (see §18) |

Avoid: "v7 five-class **taxonomy**" (v7 is a corpus/schema revision, not the taxonomy itself), "six-class live taxonomy" (the sixth class is retired), "intent subclass" (conflates two label dimensions), "8-class v7 taxonomy" (that number refers to the intent vocabulary used in correspondence backfill, not the document-class taxonomy), and `compliance_filing` described as a current pipeline class.

## Required action

Because the taxonomy is already aligned, the work here is smaller than earlier drafts assumed — it is *documentation and drift-prevention*, not *reconciliation of a gap*:

1. Remove any lingering plan/doc language (including this plan's own earlier drafts) implying a live "extended taxonomy" with unrepresented classes.
2. Point every taxonomy claim at `packages/llm-mailroom/src/pipeline/hf_corpora.py` ( `HUB_CLASSES` ) as the literal source of truth, and at `docs/v7-taxonomy.md` as the canonical prose contract.
3. Keep subclass ( `expected_subclass` ) and intent explicitly separate from document class — see §7 and §21.
4. If a future Mailroom release ever *reintroduces* `compliance_filing` , `court_opinion` , or `due_diligence` , that's a new taxonomy decision requiring its own `taxonomy_version` bump (§62) and its own §61 justification — not a "restoration" of something latent.

## 6. merger\_agreement Must Remain Distinct

`merger_agreement` is a particularly important special case. The repository treats it as an explicit classification class in the docclass evaluation surface, while it shares Contract Extraction and the contracts specialist operationally.

[confirmed] `docs/v7-taxonomy.md` lists `merger_agreement` as one of the five live



HUB\_CLASSES , with `all_cash` given as an example `expected_subclass` value — exactly the hierarchical split this section describes.

```
classification: merger_agreement
routing: contracts_specialist
extraction_schema: ContractExtraction
```

This is not contradictory — it is a hierarchical/operational distinction.

Do not merge `merger_agreement` into `contract` simply because both use the same specialist. The distinction is valuable because Mailroom may need to know “this is specifically an M&A transaction document,” while the extraction machinery can still reuse the contract schema.

---

## 7. Subtype Must Remain Distinct From Primary Class

The current sorter architecture already supports `document_type` , `contract_subtype` , `doc_subclass` . The corpus should preserve that architecture:

- **contract** → `contract_subtype`: consulting, license, franchise, ...
- **merger\_agreement** → `merger_subclass`: all\_cash, all\_stock, mixed\_cash\_stock, ...
- **corporate\_record** → `record_subclass`: bylaws, articles\_of\_incorporation, board\_resolution, ...
- **correspondence** → `correspondence_subclass`: email, letter, notice, ...
- **insurance\_claim** → `insurance_subclass`: carrier, inpatient, outpatient, pde

The sorter implementation already defines these distinctions.

---

## 8. Do Not Conflate Source With Taxonomy

Source provenance should remain explicit: `source_corpus` , `source_document_id` , `source_filename` — but must remain separate from `document_type` , `document_subtype` .

For Mailroom, source information is useful because it lets us determine:

- which source causes failures
- which formatting causes failures

- which extraction schema performs poorly
- which document families are underrepresented
- which source artifacts cause classifier shortcuts

Source is therefore an evaluation dimension, not contamination.

---

## 9. Introduce a Mailroom Document ID

Every corpus row should have a `document_id`. This becomes the stable identity of an incoming document. It must be:

- unique
- deterministic
- stable across rebuilds
- independent of row ordering
- independent of train/test split
- independent of Hub configuration

Never use a row index as the identity.

---

## 10. Keep Provenance Separate

Recommended fields:

```
document_id
source_corpus
source_document_id
source_filename
source_revision
```

The system should always be able to answer: **Where did this incoming document originate?**

---

## 11. Add Content Hashes

Add: `content_sha256`, `normalized_text_sha256`

These support duplicate detection, regression fixtures, corpus rebuilds, provenance verification, and reproducibility.

---

## 12. Duplicate Auditing Is a Mailroom Requirement

Duplicates are not necessarily errors — they are potentially important Mailroom scenarios. Classify duplicates rather than simply deleting them.

Distinguish: `exact_duplicate`, `normalized_duplicate`, `near_duplicate`,  
`template_variant`, `legitimate_repeat`

For each duplicate: `duplicate_of`, `duplicate_type` should be available.

This allows the corpus to test whether Mailroom can recognize repeated incoming documents.

---

## 13. Introduce `matter_id` / `group_id` as First-Class Ground Truth

**This is the largest dataset enhancement recommended.**

The repository already uses `session_id` / `matter_id` semantics for multi-document grouping and pilot traces. The dataset should therefore evolve toward: `document_id`, `matter_id`, `group_id` where appropriate.

Conceptually:

- **`document_id`** = individual incoming artifact
- **`matter_id`** = legal/insurance matter the document belongs to
- **`group_id`** = logical document bundle/group within the matter

Do not invent a competing grouping abstraction if the existing Mailroom `matter_id` semantics already satisfy the requirement.

---

## 14. Build Multi-Document Cases

The current corpus is primarily document-centric. The next evolution should add document bundles.

### Example — Legal:

```
MATTER-001
├─ contract.pdf
├─ amendment.pdf
├─ exhibit-a.pdf
├─ signature-page.pdf
└─ email-notice.pdf
```

### Example — Insurance:

```
MATTER-002
├─ claim_form.pdf
├─ EOB.pdf
├─ provider_bill.pdf
├─ correspondence.pdf
└─ supporting_record.pdf
```

This is much closer to the actual Mailroom problem.

---

## 14A. Matter/Group Backfill Methodology (Legacy Standalone Documents)

[new] §13–14 introduce `matter_id/group_id` as ground truth but never say how you get there from a corpus that was never collected with matters in mind.

Most of the current corpus is standalone by construction — a CUAD contract clause set or a MAUD merger agreement doesn’t arrive with sibling documents attached. Backfilling matter/group structure onto legacy rows needs an explicit, honestly-labeled method rather than a silent default:

**Source-native grouping (where it genuinely exists).** Enron correspondence has real thread/date/sender structure that can ground actual multi-document matters without fabrication — a reply chain is a legitimate `group_role: correspondence` sequence. Use this first where available, and mark it `annotation_provenance.method: source_native_thread`.

**Synthetically constructed bundles (where it doesn’t).** For standalone contracts/records with no natural sibling documents, matter/group assignment has to be manufactured — e.g. sampling a contract plus a plausible amendment/exhibit-shaped document and declaring them a matter for grouping-evaluation purposes. This is legitimate (§14 already proposes it) but **must** be flagged, not presented as if it were discovered structure:

```
grouping:
  matter_id: MATTER-00123
  group_id: GROUP-00045
  group_role: primary
  matter_construction: synthetic_constructed    # vs. source_native
```

**Do not mix the two silently.** A coverage report (§40) that counts “matters” without distinguishing source-native from synthetically constructed ones overstates how much real multi-document behavior is actually being tested. Report them as separate rows/columns.

**Sequencing.** Decide and document this methodology *before* §27–29 (simulation runs, interleaving, distractors) are built on top of it — those all assume matter/group\_id already exists and means something specific.

---

## 15. Add Document Relationships

Where ground truth permits, represent relationships rather than forcing everything into a flat class label:

```
attachment_of, exhibit_of, amendment_of, supplement_to, responds_to, references,
duplicate_of, supersedes, related_to
```

---

## 16. Add group\_role

Potential values: `primary, attachment, exhibit, amendment, supporting,`  
`correspondence, duplicate, related, unknown`

This will allow the grouping subsystem to be evaluated independently of classification.

---

## 17. Create Mailroom-Oriented Ground Truth

The existing ground truth should evolve conceptually toward:

```
ground_truth:
  classification:
    document_type:
    document_subtype:
  extraction:
    expected_fields:
```

```
grouping:
  matter_id:
  group_id:
  group_role:
  relationships:
routing:
  expected_specialist:
pipeline:
  expected_stage:
  review_expected:
  retry_expected:
```

The repository already uses `expected_subclass`, `expected_fields`, and `expected_stage` in the pilot evaluation surface. The task is therefore to extend the existing contract, not invent an entirely new one.

---

## 18. Preserve `expected_fields`

`expected_fields` should remain the canonical extraction ground truth. Do not replace it with generic `entities[]` unless the schema genuinely requires a second representation.

The current scoring system already understands field types such as `date`, `id`, `money`, `name`, `free_text`, `entity_list`, and performs deterministic field-level scoring. Enhance `expected_fields` rather than bypassing it.

[corrected] When discussing the current extraction/GT field surface, use the repo's own name for it: the **"27-key ground-truth schema"** (per `docs/v7-taxonomy.md §5`). Don't invent a new name for something that's already named.

---

## 19. Add Span-Level Evidence Where Available

For extraction tasks that have exact source evidence, preserve: `field`, `value`, `start`, `end`

Especially useful for: CUAD contract clauses, named entities, dates, monetary values, identifiers, parties.

The objective is to make extraction ground truth auditable.

---

## 20. Do Not Treat All Ground Truth as Equal

The corpus currently contains several provenance regimes. Distinguish:

```
source_native , verified_join , human_annotated , human_adjudicated , LLM_assisted ,  
LLM_zero_shot , heuristic , synthetic
```

This is particularly important for correspondence intent [corrected — see §21 for the confirmed field names; "subject\_matter" and "keywords" were an earlier guess, not verified repo vocabulary] .

The repo already has an HF ground-truth synchronization / intent-backfill script ( intent\_backfill , part of the centralized mailroom-corpus-eda upload helpers — see §44A) that generates these labels and repins the corpus revision. The provenance of those labels must remain visible.

---

## 21. Do Not Make intent Part of Primary Document Classification

Keep document\_type / expected\_subclass separate from correspondence intent. Per docs/v7-taxonomy.md , intent is carried as its own small field group, not folded into subclass:

```
intent  
intent_source  
intent_confidence  
intent_status
```

[corrected] Earlier drafts of this plan used subject\_matter and keywords as the secondary dimensions alongside intent. Those weren't confirmed against the repo — the actual confirmed fields are intent\_source , intent\_confidence , and intent\_status . If subject\_matter / keywords genuinely exist elsewhere in the schema, verify and merge them in explicitly rather than assuming they're part of the same intent group.

Example: document\_type=correspondence , expected\_subclass=attorney\_demand , intent=payment\_demand — is perfectly coherent, and matches the real subclass example ( attorney\_demand ) given in docs/v7-taxonomy.md . This is exactly the kind of orthogonality Mailroom needs.

---

## 22. Make Classification and Extraction Evaluation Cascaded

The evaluation should distinguish `classification_correct` from `routing_correct` from `extraction_correct` from `grouping_correct` from `final_pipeline_success`.

Do not collapse these into one score.

---

## 23. Measure Error Propagation

The central analysis should eventually show:

```
classification error → routing error → wrong specialist
→ extraction error → grouping/reporting error
```

This is much more informative than `accuracy = 92%`.

---

## 24. Use the Existing Mailroom Scoring System

Do not duplicate `llm-dojo-scoring`. The mono-repo already has deterministic scoring and evaluation infrastructure. The dataset should supply the canonical ground truth required by that infrastructure.

In particular, retain compatibility with: `class_correct`, `stage_correct`, `expected_field_presence`, deterministic field scores, judge scores, classification/completeness/correctness, confidence calibration.

---

## 25. Add Mailroom-Specific Corpus Metrics

The dataset report should include:

**Classification:** document-type accuracy, subtype accuracy, confidence calibration, unknown/review rate

**Extraction:** field presence, field precision, field recall, field correctness, hallucination rate

**Grouping:** matter assignment, group assignment, pairwise grouping precision, pairwise grouping recall, relationship accuracy

**Operational:** review rate, retry rate, first-pass success, final success, irrecoverable failure

**Cost:** LLM calls, input tokens, output tokens, estimated cost, latency



These map directly onto the Mailroom architecture.

---

## 26. The Corpus Should Have Two Modes

**Mode A — Isolated Document:** One document enters Mailroom. Used for classifier testing, specialist testing, extraction testing, regression tests.

**Mode B — Ingress Stream:** Multiple documents enter Mailroom. Used for grouping, matter reconstruction, duplicate detection, interleaving, routing, concurrency, multi-document reasoning.

The same underlying document should be reusable in both modes.

---

## 27. Introduce `simulation_run_id`

For stream experiments: `simulation_run_id`

Example:

```
RUN-001
DOC-01 → Matter A
DOC-02 → Matter B
DOC-03 → Matter A
DOC-04 → unrelated
DOC-05 → Matter B
```

This lets the exact incoming sequence be reproduced.

---

## 28. Support Interleaved Matters

The corpus should eventually be capable of generating: `A1 B1 A2 C1 B2 A3 C2` rather than `A1 A2 A3 B1 B2 C1 C2`.

The former tests the actual grouping problem.

---

## 29. Add Distractors

A grouping scenario should contain documents that do not belong to the target matter.

Matter A: contract, amendment, exhibit

Matter B: claim, EOB

Distractors: unrelated email, unrelated corporate record

This tests whether grouping relies on meaningful evidence rather than proximity.

---

## 30. Add Controlled Failure Scenarios

The corpus should eventually explicitly encode:

```
normal, ambiguous, unknown, duplicate, missing_information,
conflicting_information, misleading_metadata, low_quality_text, multi_class,
wrong_initial_classification, wrong_initial_extraction
```

These are not “bad data” — they are Mailroom test fixtures.

---

## 31. Add Review/Retry Ground Truth

The corpus should support: review\_expected, review\_reason, correction\_available, retry\_expected, expected\_post\_retry\_state

Example:

```
review_expected: true
review_reason: incomplete_extraction
retry_expected: true
expected_post_retry_state: archived
```

This aligns directly with the Mailroom arbiter/review/retry architecture.

---

## 32. Create Regression Fixtures From Real Failures

This should become a permanent feedback loop:

```
Mailroom failure → diagnosis → corrected ground truth
→ dataset fixture → regression test → future Mailroom release
```

Every significant production/pilot failure should have the opportunity to become a

permanent regression case. This is arguably the most valuable long-term function of docclass-merged.

---

### 33. Do Not Balance the Dataset Artificially

The current class mixture should remain useful as an ingress corpus. Do not force 20% each unless that represents actual Mailroom traffic.

Instead maintain three distinct distributions:

- **Production-like distribution** — representative incoming traffic
- **Diagnostic distribution** — enough examples to exercise every class/subclass
- **Challenge distribution** — intentionally difficult cases

These are different purposes.

---

### 34. Expand by Mailroom Coverage, Not Row Count

The next documents added should answer: **What Mailroom behavior is currently untested?**

Priority should be based on: taxonomy coverage, specialist coverage, field coverage, grouping coverage, failure-mode coverage, source/format coverage — not "+1,000 documents."

---

### 35. Legal Expansion Priorities

Prioritize document families that Mailroom actually needs. Potential additions:

amendments, exhibits, schedules, side letters, notices, signature pages, employment agreements, vendor agreements, NDA variants, licensing agreements, purchase agreements, corporate resolutions, board consents, powers of attorney

Do not create new classes unless Mailroom actually needs to route them differently. Some should remain `document_type=contract`, `document_subtype=...` rather than becoming new primary classes.

---

## 36. Insurance Expansion Priorities

The current insurance corpus should be expanded in workflow diversity, not just row count. Prioritize:

FNOL, EOB, carrier correspondence, coverage determination, denial, reservation of rights, adjuster report, demand package, provider bill, claim form, supporting documentation

The existing taxonomy already describes insurance claims broadly in these terms.

---

## 37. Correspondence Should Become Cross-Cutting

Correspondence should not merely be an Enron corpus. It should eventually serve as: legal correspondence, insurance correspondence, corporate correspondence, transaction correspondence — while retaining `communication_type`, `intent`, `subject_matter` as secondary dimensions.

---

## 38. Corporate Records Should Be Expanded

The current class is comparatively small. Prioritize:

bylaws, articles, charter amendments, board resolutions, officer certificates, powers of attorney, subsidiary lists, rights instruments, indentures

The sorter already has this taxonomy. Expansion should improve subclass coverage, not simply increase `corporate_record` count.

---

## 39. Retain Synthetic Insurance Data

Do not remove CMS DE-SynPUF simply because it is synthetic. It is useful for deterministic regression, predictable structure, extraction scoring, repeatable pipeline tests, and stress testing. Explicitly identify its synthetic provenance. Eventually add more heterogeneous insurance sources/templates.

---

## 40. Create a Mailroom Corpus Coverage Matrix

Create: `docs/reports/audits/docclass_coverage_matrix.md`

**Rows:** document\_type, document\_subtype, source, specialist, fields, scenario **Columns:** available, ground truth, tested, regression, challenge, multi-document

This becomes the primary answer to: **What parts of Mailroom have actually been tested?**

---

## 41. Add Field-Coverage Analysis

For every specialist, track: expected field, number of examples, source coverage, document subtype coverage, ground-truth quality, difficulty.

Example (insurance\_claim): claim\_number 500, policy\_number 480, adjuster 72, denial\_reasons 31, coverage\_determination 600

This immediately reveals which extraction fields lack meaningful evaluation coverage.

---

## 42. Do Not Overstate Ground-Truth Quality

Especially for automatically generated fields. The dataset card/report should distinguish: classification ground truth, extraction ground truth, secondary enrichment, LLM-derived annotation, synthetic values.

A field generated by an LLM should not silently become indistinguishable from a manually verified field.

---

## 43. Add annotation\_provenance

```
annotation_provenance:
  source:
  method:
  model:
  prompt_version:
  confidence:
  reviewer:
  timestamp:
```

This becomes particularly important because the mono-repo already treats prompt lineage and evaluation methodology as first-class artifacts.

---

## 44. Pin the Corpus Revision

The repo already has `FULL_CORPUS_REVISION` behavior in its HF synchronization tooling. Make this an explicit release invariant.

```
dataset:
  name: Lucius-Morningstar/docclass-merged
  revision: <immutable revision>
```

Never evaluate against an unpinned main/latest dataset.

---

### 44A. HF Publishing Checklist — Centralized Upload Path

[new, grounded in AGENTS.md] The repo already has a firm rule here that this plan needs to inherit rather than re-derive: **the docclass-merged family is published only through the centralized helpers in `packages/mailroom-corpus-eda/src/mailroom_eda/` — `hf_interface`, `dataset_export`, `docclass_uploader`, `intent_backfill` — never ad-hoc upload code.**

Before any pinned revision (§44) is actually pushed, confirm the publish path covers:

- cast-safe metadata (no silent type coercion on upload)
- line-boundary-safe JSONL (no row corruption from naive line splitting)
- sha256 verification post-upload (ties directly into §11's content hashes)
- surgical dataset-card renders (the card updates only the sections that changed, not a full regeneration that could clobber hand-written context)
- the blind-config label guard (prevents a metadata-ablation view from accidentally becoming a labeled/promoted config — directly relevant to §47's warning against a premature blind config)
- intent hydration status (tracked against the repo's own issue #5 — confirm current state before assuming intent fields are or aren't backfilled)

Also confirm standard HF dataset-card requirements are met before the revision is publishable: YAML frontmatter with `license`, `task_categories`, and `size_categories` set correctly for a five-class, multi-source, non-training corpus (§78's README paragraph should live inside this frontmatter-described card, not replace it).

Any agent tempted to write a quick upload script directly against `huggingface_hub` for

expediency should stop and find the existing helper instead — see the repo's own `huggingface` `opencode` skill for the full workflow.

---

## 45. Make Dataset Revision Part of the Trace

Every Mailroom evaluation trace should record: `dataset_name`, `dataset_revision`, `document_id`, `matter_id`, `simulation_run_id`, `taxonomy_version`

---

## 46. Make Prompt + Dataset + Model a Reproducible Triple

Every experiment should identify: dataset revision + taxonomy revision + prompt version + model/provider + configuration. This creates a reproducible experiment identity.

---

## 47. Do Not Create a Generic “Blind” Config Yet

A blind ML benchmark surface is not the immediate priority. Mailroom needs provenance and realistic ingress conditions. If a metadata-ablation experiment is useful, implement it as an evaluation transform, not necessarily as a permanent dataset configuration.

Example experimental views: full ingress, metadata-stripped, filename-stripped, source-stripped. Do not complicate the canonical corpus unnecessarily.

---

## 48. Do Create an Isolated vs. Ingress Evaluation Split

This is more useful to Mailroom than train/validation/test:

- **DOCUMENT EVAL** → single-document behavior
  - **MATTER EVAL** → multi-document grouping
  - **STREAM EVAL** → interleaved incoming documents
  - **RECOVERY EVAL** → review → correction → retry
- 

## 49. Reframe Train/Test Splits

The dataset does not need conventional ML train/test semantics as its organizing principle. If splits remain for sampling/reproducibility, document them as **evaluation partitions** rather than implying model-training methodology. The primary unit should be an **evaluation scenario**, not a training example.

---

## 50. Create Four Evaluation Tiers

- **Tier 1 — Document:** one document, classification, subtype, extraction
- **Tier 2 — Matter:** related documents, grouping, relationships
- **Tier 3 — Stream:** multiple matters, interleaving, distractors, duplicates
- **Tier 4 — Recovery:** initial failure, review, correction, retry, final success

This should become the Mailroom evaluation ladder.

---

## 51. Create Difficulty Levels

- L0 — clean single document
- L1 — subtype ambiguity
- L2 — extraction complexity
- L3 — multi-document matter
- L4 — interleaved matters
- L5 — distractors/duplicates
- L6 — ambiguous/conflicting documents
- L7 — adjudication + retry

The exact naming is flexible; the concept is not.

---

## 52. Make Regression the Center of the Dataset

The mature corpus should contain: gold examples + normal examples + edge cases + challenge cases + known failures + repaired failures. The last two categories are extremely important.



---

## 53. EDA Should Shift From “Dataset Balance” to “System Coverage”

The EDA report should answer:

- **What documents?** class/subclass/source
- **What extraction?** field coverage, entity density
- **What grouping?** matter/group size, relationships
- **What difficulty?** length, format, ambiguity
- **What failures?** classification, extraction, grouping, review, retry
- **What sources?** corpus/source/template

---

## 54. Add Pipeline-Relevant Text Metrics

Continue collecting character count, word count, token count, page count — but interpret them as pipeline stress variables. E.g., very long contract, scanned EOB, email thread, multi-page corporate exhibit should become meaningful stress categories.

---

## 55. Format Should Be First-Class

Track: `pdf_text`, `pdf_scanned`, `image`, `email`, `html`, `txt`, `table-heavy`, `mixed`

Mailroom’s ingestion layer explicitly has PDF transcription and image/vision handling; vision is additive to the complete text representation. Format diversity should be part of the evaluation corpus.

---

## 56. Create Ingestion-Stress Cases

Include: text PDF, scanned PDF, OCR-heavy PDF, image document, table-heavy PDF, long document, short document, malformed document, mixed page layouts.

This tests the actual front door of Mailroom.

---

## 57. Preserve Raw Artifacts Where Legally Appropriate

The simulation should ideally retain: original artifact + extracted text + expected extraction — otherwise it becomes impossible to distinguish ingestion failure from LLM classification failure from specialist extraction failure. That distinction is critical.

---

## 58. Add Stage-of-Failure Ground Truth

For known regression scenarios:

```
failure_stage: ingestion | classification | routing | extraction  
               | validation | grouping | adjudication | archival
```

This lets the evaluation system measure where Mailroom actually broke.

---

## 59. Add Expected Routing

Where appropriate: `expected_specialist`

Example: `merger_agreement → contracts_specialist`, `insurance_claim → insurance_claims_specialist`

The repository already has this mapping in its taxonomy/specialist architecture.

---

## 60. Do Not Resurrect Retired Classes Without a New Taxonomy Decision

[corrected] Earlier drafts of this section described `court_opinion`, `due_diligence`, and `compliance_filing` as living in a “broader Mailroom taxonomy” that the corpus simply hadn’t caught up to yet. Per `docs/v7-taxonomy.md`, that’s not the current state:

`compliance_filing` has been **retired** from the pipeline, and `court` / `dd` are **retired** and are not v7 classes. There is no broader live taxonomy currently holding them — they’re former classes.

**Rule:** a retired class is not a future corpus class waiting on data. If Mailroom needs one of these document families again, that’s a fresh taxonomy decision (new `taxonomy_version`, new `HUB_CLASSES` entry, updated `docs/v7-taxonomy.md`, updated `mailroom-pipeline.svg`)

— not a reconciliation of an existing gap. Historical references to these three in changelogs, archived fixtures, or migration history describe the former system and are not current pipeline vocabulary; don't let them leak into new documentation as if they were live.

---

## 61. Future Corpus Expansion Should Follow This Rule

Add a new document family only when at least one of these is true:

1. Mailroom needs to route it differently.
2. It requires a different extraction schema.
3. It creates a meaningful grouping distinction.
4. It represents a known operational failure mode.
5. It is required for a production workflow.

Do not add data merely to increase corpus size.

---

## 62. Version Taxonomy and Dataset Independently

Use: `dataset_version`, `taxonomy_version`, `annotation_version`, `evaluation_version`

A prompt change should not require a dataset version. A label correction should. A new evaluation methodology should not necessarily require either.

---

## 63. Add Dataset Contract Tests

Create tests that assert: `document_id` unique, `document_type` valid, `document_subtype` valid, `subtype` belongs to `type`, `expected_fields` schema-valid, `expected_stage` valid, `source` provenance present, `annotation` provenance valid, `content` hash valid.

---

## 64. Add Mailroom Contract Tests

The dataset should validate against the actual Mailroom interfaces. For every corpus class: `classification` schema, `specialist` schema, `routing` target, `expected_fields`, `field` scorer, `judge` contract must agree.

This prevents taxonomy drift between HF, Sorter, Mailroom, Dojo, Sandbox.

---

## 65. Make Taxonomy Drift Impossible

The mono-repo already contains multiple taxonomy consumers. The agent should establish a canonical source and automated parity checks. At minimum validate: `taxonomy.yaml` , sorter classes, specialist registry, HF class universe, scoring class universe, sandbox class universe, documentation.

The test should fail when these disagree unexpectedly.

---

### 65A. Taxonomy Drift Enforcement — Live CI Gates

[new, grounded in AGENTS.md/README.md] §65's "canonical source and automated parity checks" already has real infrastructure to hook into rather than invent:

- **Board/governance CI:** `.github/workflows/board-governance.yml` runs `scripts/board_state.py` check on every change to `governance/` , `scripts/` , or `.github/` — structural contradictions exit 1. A taxonomy-parity check belongs in the same enforcement family, not a separate ad-hoc script.
- **Label taxonomy drift:** `scripts/github_labels.py` `audit` is an existing CI gate for a *different* taxonomy (issue labels) — model the document-class parity check on this pattern (a single `audit / check` command, non-zero exit on drift) rather than a bespoke one-off.
- **Literal source of truth:** parity checks should diff against `HUB_CLASSES` in `packages/llm-mailroom/src/pipeline/hf_corpora.py` directly, not a hand-maintained copy of the class list.

Concretely: a taxonomy-parity check should fail CI if `HUB_CLASSES` , the HF corpus's represented classes, the sorter's output vocabulary, the specialist registry, and `docs/v7-taxonomy.md` 's stated five classes ever disagree — surfaced the same way board invariant failures are (exit 1, readable diagnostic, blocks merge).

---

## 66. Specifically Test the Current Five-Class GT

The canonical docclass-merged GT — the **"v7 five-class corpus"** in the repo's own naming

(§5) — should currently validate to: `contract`, `corporate_record`, `correspondence`, `insurance_claim`, `merger_agreement`

[corrected] Anything else should be explicitly classified as retired ( `compliance_filing`, `court_opinion`, `due_diligence` — see §60) or `unknown` — not as “extended taxonomy,” since no live extended taxonomy currently exists beyond these five.

---

## 67. Keep unknown Semantically Important

`unknown` is not a dataset failure. It represents: *Mailroom cannot safely map this document to an available operational class*. That should route to review rather than forcing a false classification.

The repository already uses this behavior for unsupported/retired document types.

---

## 68. Build an Unknown/OOD Collection

Later, deliberately add documents that should produce `unknown` rather than adding them to one of the five classes. This will test whether the sorter knows when not to guess.

---

## 69. Add Confidence-Band Coverage

The current Mailroom routing thresholds are:

```
>= 0.95      → continue
0.70 - 0.95  → review/reviewer pathway
< 0.70      → retry
```

with retry limits defined in taxonomy configuration.

The corpus should deliberately contain examples occupying high-confidence, medium-confidence, low-confidence, and unknown bands. This is much more valuable than merely maximizing classification accuracy.

---

## 70. Create Confidence-Calibration Fixtures

The dataset should contain cases where: correct + high confidence, correct + low

confidence, wrong + high confidence, wrong + low confidence. This lets Mailroom test whether its routing policy is actually sensible.

---

## 71. Add Review-Routing Coverage

The corpus should explicitly test: high confidence → no review; medium confidence → reviewer/review; low confidence → retry; low after retry → human review. This should be a regression suite.

---

## 72. Add Arbiter Cases

The current architecture already supports: judge → arbiter → stand / re-extract / human. The dataset should eventually contain fixtures for all of these, e.g.:

- partial extraction → arbiter orders re-extraction → extraction improves → archive
  - partial extraction → arbiter cannot resolve → human review
- 

## 73. Add Human-Review Cases

Represent: `review_expected`, `review_reason`, `review_resolution`. This allows the dataset to test the human-in-the-loop contract rather than treating review as an external event.

---

## 74. Make Retry a First-Class Outcome

Do not only record `initial_result`. Record: `attempt_1`, `attempt_2`, `review`, `correction`, `attempt_3`, `final_result` where applicable. The point is to evaluate whether Mailroom recovers, not merely whether it initially succeeds.

---

## 75. Create a Mailroom Recovery Score

Eventually report `first_pass_success` and `final_success_after_recovery` — these answer different questions.

Example: First pass: 84%. After retry/review: 96%. That tells you the adjudication

architecture is contributing value.

---

## 76. Add Cost-Aware Pipeline Evaluation

The corpus already supports cost/LLM call tracking. For every scenario report: classification calls, extraction calls, judge calls, arbiter calls, retry count, input tokens, output tokens, estimated cost, latency. Then compare accuracy vs. cost vs. recovery.

---

## 77. Do Not Turn Every Ground-Truth Label Into a Judge Task

Use the existing deterministic field scorer wherever possible. The repository already has type-aware scoring and only escalates ambiguous cases to the LLM judge. The corpus should preserve enough structured ground truth for deterministic scoring to remain authoritative.

---

## 78. Create a Mailroom Corpus README Section

The dataset documentation should explicitly state:

docclass-merged is an evaluation/ingress simulation corpus for LLM-Mailroom. It is not intended as a model-training corpus and its component corpora are deliberately heterogeneous. The primary evaluation target is end-to-end Mailroom behavior across classification, specialist routing, extraction, grouping, validation, adjudication, retry, and archival.

This should be the first conceptual paragraph of the dataset documentation.

---

## 79. Update the Mono-Repo Documentation

Because this mono-repo is the canonical documentation surface, changes should be reflected in: `packages/llm-mailroom/docs/` , `packages/llm-entity-extraction/docs/` , `packages/llm-dojo-scoring/docs/` , `docs/sister-repos.md` , dataset-related reports.

Do not fix the HF dataset documentation while leaving the mono-repo's terminology inconsistent.

---

## 80. Create One Canonical Dataset Contract

Create: docs/DOCCLASS\_CONTRACT.md

It should define: What is docclass-merged? What is it not? What classes does it contain? What classes exist only in the extended taxonomy? What is a document? What is a matter? What is a group? What is expected\_fields? What is provenance? What constitutes ground truth? How is it evaluated? How does it map into Mailroom?

This becomes the authoritative bridge between the dataset and pipeline.

---

## 81. Create a Dataset → Mailroom Mapping Table

| Dataset concept   | Mailroom concept              |
|-------------------|-------------------------------|
| document_id       | document identity             |
| matter_id         | matter/session grouping       |
| document_type     | sorter output                 |
| document_subtype  | sorter subtype                |
| expected_fields   | specialist ground truth       |
| source_corpus     | provenance                    |
| simulation_run_id | evaluation session            |
| expected_stage    | pipeline terminal expectation |
| review_expected   | review routing                |
| retry_expected    | recovery behavior             |
| relationships     | document grouping/association |

This should live in the canonical docs.

---

## 82. Build the Corpus Around Five Current Domains

Current merged-corpus scope: **contract, corporate\_record, correspondence,**



## insurance\_claim, merger\_agreement

[corrected] These aren't just "the central working universe" pending a larger taxonomy — per §5, this is currently the **complete** live Mailroom taxonomy. Future expansion (§61) means adding subclasses/format/matter coverage within these five, or making a fresh, deliberate taxonomy decision to add a sixth class — not catching up to classes that already exist elsewhere.

---

## 83. The Corpus Should Not Be Forced to Become "Benchmark Balanced"

Instead evaluate **coverage** across: class, subclass, source, format, field, matter, scenario, failure mode. This is the correct notion of completeness for Mailroom.

---

## 84. Release Structure

- v0.1-working — current corpus snapshot
- v0.2-mailroom-hardened — identity + provenance + taxonomy + evaluation contract
- v0.3-matter-aware — grouping + relationships + multi-document cases
- v0.4-recovery-suite — review + retry + arbiter scenarios
- v1.0-mailroom-evaluation — stable regression/evaluation corpus

These are conceptual targets, not requirements to artificially manufacture releases.

---

## 84A. Phase Dependency Graph & Sequencing

[new] P0–P5 (§85–90) implies an order by number but never states hard prerequisites, which invites parallelizing work that actually has dependencies. The real chain, reading top-down from what blocks what:

```
document_id (P0)
└─> content_sha256 / normalized_text_sha256 (P0)
    └─> duplicate_classification (P0/P2)
└─> annotation_provenance (P1)
    └─> confidence-band + calibration_fixtures (P1)
```

```

matter_id / group_id backfill method decided (§14A)
  ↳ group_role, relationships (P2)
    ↳ multi-document cases, interleaving, distractors (P2)
      ↳ simulation_run_id, stream eval (P2/P3)
failure_stage + review_expected/retry_expected ground truth (P1)
  ↳ arbiter/human-review fixtures (P3)
    ↳ first-pass vs. recovered success scoring (P3/P5)
taxonomy parity CI gate (§65, §65A)
  ↳ must be green before any P2+ schema change ships — matter/group additions

```

This mirrors the real corpus-feed → prompt-loop → scoring → pipeline → surfaces dependency chain the monorepo is structured around (see the architecture diagram in `README.md`): corpus work (this plan) feeds the prompt-experiment loop, which feeds `llm-dojo-scoring`, which `llm-mailroom` imports — schema changes here ripple forward through all of it, so §65A's parity gate has to hold at each step.

## 84B. Definition-of-Done Contract (No Silent Completion)

[new, grounded in `AGENTS.md`] Individual checklist items in §85–90 read as intentions ("establish stable `document_id`") rather than testable outcomes. The monorepo already has a stricter, board-enforced definition of done that every item in this plan should inherit:

A task is **not** done until, per `AGENTS.md`'s "no silent completion" rule:

- the surgically relevant test suite is green for every touched package (full suite instead, if the change is cross-cutting — e.g. a schema field addition touches `mailroom-corpus-eda`, `llm-dojo-scoring`, and `llm-mailroom` alike)
- `git status` is clean for the card's scope
- Evidence names the actual commit(s)
- for synced cards, the linked GitHub issue is closed in the same commit
- any doc whose described behavior changed (`README.md`, `AGENTS.md`, `governance/TASKS.md`, `docs/v7-taxonomy.md`) is updated in the same commit

Concretely, rewrite each P0–P5 checkbox as a claim plus a check, e.g.:

- ~~"Establish stable `document_id`"~~ → "100% of rows have a unique, deterministic `document_id`, verified by the §63 contract test; card moves to `done` only once that test is green in CI and referenced in Evidence."
- ~~"Pin HF corpus revision"~~ → "Dataset revision is pinned and referenced in at least one evaluation trace (§45); card cites the commit that changed the pin."

Work this way through §85–90 rather than treating a checkbox as complete once the underlying field merely exists in the schema.

---

## **85. P0 — Implement Immediately**

- Freeze current dataset revision
  - Create baseline report
  - Define canonical dataset contract
  - Reconcile five-class GT with extended Mailroom taxonomy
  - Explicitly document merger\_agreement → contracts specialist behavior
  - Establish stable document\_id
  - Preserve source provenance
  - Add content hashes
  - Validate document\_type/subtype compatibility
  - Validate expected\_fields against specialist schemas
  - Pin HF corpus revision
  - Record dataset revision in evaluation metadata
  - Add taxonomy parity tests
  - Correct stale dataset/mono-repo documentation
- 

## **86. P1 — Mailroom Evaluation Hardening**

- Formalize expected\_fields
- Preserve evidence spans where available
- Formalize annotation provenance
- Add expected routing
- Add expected stage
- Add review\_expected

- Add retry\_expected
  - Add recovery expectations
  - Add unknown/OOD fixtures
  - Add confidence-band fixtures
  - Build class/subclass/field coverage report
- 

## **87. P2 — Matter/Grouping**

- Introduce/reuse matter\_id
  - Introduce group\_id where necessary
  - Add group\_role
  - Add document relationships
  - Build multi-document cases
  - Build interleaved cases
  - Add distractors
  - Add duplicate/repeated documents
- 

## **88. P3 — Recovery**

- Create intentional failure fixtures
  - Add review corrections
  - Add retry ground truth
  - Add arbiter scenarios
  - Add human-review scenarios
  - Measure first-pass vs. recovered success
  - Promote significant Mailroom failures into regression cases
-

## 89. P4 — Corpus Expansion

Expand according to Mailroom needs:

- Legal document families
- Insurance workflow documents
- Correspondence contexts
- Corporate records
- Contract subclasses
- Merger documents
- Format diversity
- Grouping scenarios
- Adversarial/ambiguous cases

Do not optimize for raw row count.

---

## 90. P5 — Advanced Evaluation

Eventually:

- Source-specific failure analysis
  - Format-specific failure analysis
  - Confidence calibration
  - Cost/accuracy tradeoffs
  - Recovery-value analysis
  - Grouping metrics
  - Relationship metrics
  - Stream-level evaluation
  - OOD/unknown detection
  - Longitudinal regression tracking
-

## 91. Release Gates

A Mailroom corpus release should fail validation if:

- `document_id` is not unique
- taxonomy value is invalid
- subtype does not belong to document type
- `expected_fields` violates specialist schema
- required provenance is missing
- dataset revision is not pinned
- ground-truth annotation provenance is missing
- source/document identity is ambiguous

For matter-aware releases, additionally:

- `group_id` invalid
- relationship references missing document
- matter references missing document
- simulation sequence is not reproducible

For recovery releases:

- retry scenario has no expected outcome
- review correction has no provenance
- failure scenario cannot be reproduced

---

### 91A. Out of Scope

[new] To keep this dataset-hardening effort from scope-creeping into a platform rewrite, grounded in `AGENTS.md` 's workspace rules:

- **Does not** modify `llm-mailroom` 's LangGraph pipeline logic itself (nodes, routing thresholds, arbiter behavior) — this plan supplies ground truth *for* that pipeline, it doesn't redesign it.
- **Does not** touch `The-Mailroom visualizer`, `agent-mailroom`, or `local-mailroom-`

`sandbox` internals beyond whatever new fields the schema changes require them to read.

- **Does not** retrain or fine-tune any classifier or extraction model — this is a ground-truth/evaluation-corpus effort, not a training effort (§1, §93).
- **Does not** delete or bypass published dependency pin lines in any package `pyproject.toml` — per `AGENTS.md`, dev-time redirection happens only through `[tool.uv.sources]`, and pins are bumped only at release time.
- **Does not** write ad-hoc HF upload code — all publishing goes through the centralized helpers (§44A). An agent finding it faster to call `huggingface_hub` directly should treat that urge as a sign to go find the existing helper, not a shortcut to take.
- **Does not** prune the `mailroom-corpus-eda` EDA deliverables ( `reports/figures/`, `reports/figures_interactive/`, `reports/tables/`, `SUMMARY_REPORT.*` ) — these are tracked in full per human directive (HUB-008), unlike other heavy assets in the repo.
- **Does not** reintroduce `compliance_filing`, `court_opinion`, or `due_diligence` as live classes (§60) without a separate, explicit taxonomy decision outside this plan's scope.

---

## 92. The Central Dataset Object Should Eventually Look Like This

Conceptually (not a mandate to physically serialize as nested YAML/JSON — existing schemas and interfaces should determine the implementation):

```
document:
  document_id: DOC-000123
  content:
    raw:
      extracted_text:
  provenance:
    source_corpus:
    source_document_id:
    source_filename:
    source_revision:
  classification:
    document_type: insurance_claim
    document_subtype: outpatient
    confidence_target:
  extraction:
    expected_fields:
```

```
    claim_number:
    policy_number:
    insurer:
    claimed_amount:
    coverage_determination:
grouping:
    matter_id:
    group_id:
    group_role:
    relationships: []
simulation:
    simulation_run_id:
    sequence_position:
evaluation:
    expected_route:
    expected_stage:
    review_expected:
    retry_expected:
    expected_final_state:
annotation:
    source:
    method:
    version:
    confidence:
```

---

## 93. Final Agent Directive

Treat docclass-merged as the canonical Mailroom ingress/evaluation corpus, **not** as a model-training dataset. Preserve its heterogeneous fusion because that heterogeneity intentionally simulates the incoming document universe Mailroom must process.

Do not optimize the corpus for conventional ML benchmark purity, artificial class balance, or source independence. Optimize it for operational coverage of the Mailroom pipeline.

The current five-class ground-truth universe is `contract`, `corporate_record`, `correspondence`, `insurance_claim`, and `merger_agreement` — this is now the **complete** five-class live Mailroom taxonomy, not a subset of a larger one ( `[corrected]` , see §5). `due_diligence`, `compliance_filing`, and `court_opinion` are retired former classes, not extended/unsupported classes awaiting corpus coverage; keep them out of live documentation except as explicit historical/changelog references (§60).

Preserve `merger_agreement` as a distinct classification label even though it operationally routes to the contracts specialist and shares Contract Extraction. Classification taxonomy



and specialist implementation are separate concerns.

Harden the corpus around stable document identity, provenance, taxonomy parity, expected extraction fields, annotation provenance, source/format metadata, and pinned dataset revisions. Reuse the existing Mailroom/dojo evaluation infrastructure rather than creating parallel scoring systems.

Extend the ground-truth contract from isolated document classification toward Mailroom evaluation: document type, subtype, expected fields, expected routing, expected stage, matter/group membership, relationships, review expectations, retry expectations, and final expected state.

Build multi-document matter/group scenarios after the document-level contract is stable. The mature corpus should support isolated documents, multi-document matters, interleaved document streams, distractors, duplicates, ambiguous documents, and controlled failure/recovery cases.

Treat every important Mailroom failure as a candidate regression fixture. The corpus should become a continuously expanding regression/evaluation substrate for the entire system rather than a static collection of examples.

Prioritize legal and insurance workflow coverage. Expand the corpus according to Mailroom's actual routing, extraction, grouping, and recovery requirements — not according to arbitrary document count.

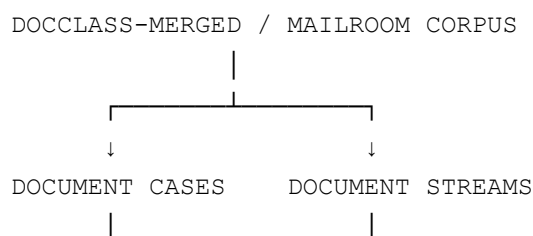
Every experiment must be reproducible from the tuple: **dataset revision + taxonomy revision + prompt version + model/provider + runtime configuration**.

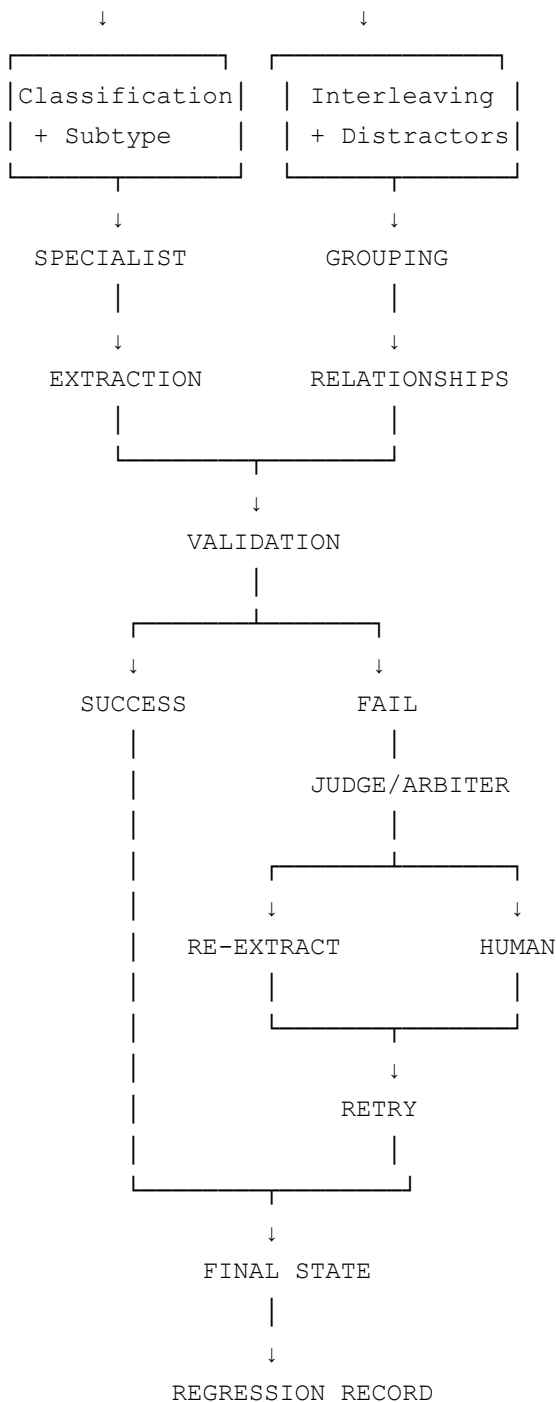
The ultimate question this corpus must answer is not "How accurately does a model classify these documents?" It is:

**"Given this incoming document or document stream, does Mailroom correctly ingest, classify, route, extract, group, validate, adjudicate, retry when necessary, and reach the correct final state?"**

---

## 94. The Strategic End State





That is the direction now considered canonical for docclass-merged.

The crucial realization from reviewing the mono-repo is that the project is much further along architecturally than the standalone HF dataset makes apparent. The repository already has the beginnings of this contract: `expected_fields`, `expected_stage`, `session_id` /matter semantics, deterministic field scoring, judge/arbiter pathways, taxonomy-driven routing, HF synchronization, and a substantial evaluation suite.

So the agent's job is not to invent a new dataset system. It is to make docclass-merged the clean, explicit, versioned data surface that the Mailroom architecture already built is

implicitly asking for.

---

## 95. Revision Note — This Enhancement Pass

This revision adds eight structural sections (§0 glossary, §4A data governance & licensing, §14A matter/group backfill methodology, §44A HF publishing checklist, §65A CI drift-enforcement, §84A phase dependency graph, §84B definition-of-done contract, §91A out-of-scope) and corrects several taxonomy claims against the live repo ( `README.md` , `AGENTS.md` , `docs/v7-taxonomy.md` , fetched 2026-09-02):

- The “extended Mailroom taxonomy” premise in §5/§60/§66/§82/§93 was wrong — the live taxonomy and the v7 corpus are already aligned at exactly five classes; `compliance_filing` , `court_opinion` , and `due_diligence` are retired, not pending.
- Correspondence intent fields (§20–21) are `intent` / `intent_source` / `intent_confidence` / `intent_status` , not the previously guessed `subject_matter` / `keywords` .
- The centralized HF publishing path ( `packages/mailroom-corpus-eda` ) was missing from the original repository inventory (§2) entirely.

Sections marked `[corrected]` replace a specific earlier claim; sections marked `[new]` are additions with no prior counterpart. Anything unflagged elsewhere in the document is unchanged from the original draft and has not been independently re-verified against the live repo in this pass — treat it as a recommendation, not a confirmed fact, until it has been.